

Manuel technique

Architecture, modèle de données, API, sécurité et déploiement.

Police Bagage est un système d'embarquement et de lutte contre la fraude bagage, déployé à l'Aéroport International de Kinshasa (FIH) sur les vols Air Congo.

Le problème traité : au comptoir d'enregistrement, une étiquette peut être émise sur un passager qui n'a déclaré aucun bagage. Le colis part alors en soute sans contrôle. Le système relie chaque bagage à un passager réellement enregistré et intercepte les colis non déclarés avant le chargement, avec alerte immédiate du superviseur.

Ce document s'adresse aux développeurs et aux exploitants du système. Pour l'usage quotidien des portails, voir les manuels d'utilisation.

1. Architecture

Le projet est un monorepo Turborepo regroupant cinq applications et trois paquets partagés.

apps/		
mobile/	React Native + Expo	agents terrain sur PDA Zebra
web/	Next.js	dashboard superviseur et admin
vols/	Next.js	tableau des vols public
litige/	Next.js	gestion des litiges bagage
tracking/	Next.js	suivi bagage passager public
packages/		
api/	Fastify	logique anti-fraude, routes de scan
bcbp-parser/	TypeScript	parser boarding pass et etiquette
shared/	TypeScript	types et constantes partagés
supabase/		
migrations/	SQL	schema, RLS, realtime, fonctions

Couches techniques

Couche	Technologie	Remarque
Mobile	React Native 0.76, Expo SDK 52	Android uniquement, PDA Zebra
Web	Next.js 15 App Router	Quatre portails distincts
API	Node 20+, Fastify 5	Processus unique, clé service_role
Base, Auth, Realtime	Supabase (PostgreSQL 17)	RLS activée sur toutes les tables
Parser BCBP	librairie npm bcbp v6	Standard IATA
Parser étiquette	fonction interne	Format 10 ou 13 chiffres

Aucune dépendance à un modèle de langage. Le BCBP est un standard documenté, l'étiquette bagage est parsée par une fonction interne.

a

2. Modèle de données

Huit tables dans le schéma public. Les colonnes marquantes pour la logique métier sont détaillées ci-dessous.

Table	Rôle	Colonnes clés
profiles	Comptes et rôles	role (admin, supervisor, agent), gate, airport_code
flights	Vols du jour	flight_number, origin, destination, stops, status, date
passengers	Passagers enregistrés	pnr, seat, declared_baggage_count, boarded
passenger_legs	Escales par passager	origin, destination, leg_order
baggage	Bagages	tag_number, serial_number, is_confirmed, on_dolly, in_hold, rush, soute
fraud_alerts	Alertes de rejet	tag_number, reason, gate, resolved
baggage_disputes	Litiges et réclamations	status, from_passenger, notes
airline_codes	Codes compagnies	numeric_code, iata_code

Clé de liaison passager et bagage

La liaison ne se fait jamais sur le code compagnie. Le code numérique 071 correspond à la fois à Ethiopian Airlines et à Air Congo, ce qui est normal et voulu. La clé est toujours le numéro de série sur 6 chiffres, associé au vol et à la date.

```

4 071 303791 002
| | | +-- 3 chiffres : nombre de bagages consecutifs (etiquette boarding)
| | +----- 6 chiffres : numero de serie, cle de liaison
| +----- 3 chiffres : code numerique compagnie
+----- 1 chiffre : Baggage Tag Issuer Code

```

L'étiquette physique scannée au tapis fait 10 chiffres. L'étiquette portée par le boarding pass en fait 13 : les 3 derniers indiquent combien de bagages consécutifs sont couverts. Au check-in, l'API déplie cette plage en autant d'étiquettes physiques.

a

3. API de scan

Sept routes exposées par le service Fastify. Toutes les routes de scan exigent un jeton d'authentification valide.

Route	Usage	Effet
POST /scan/boarding	Check-in passager	Crée le passager, ses escales, et pré-enregistre ses bagages
POST /scan/baggage	Étiquette au tapis	Applique les 5 règles anti-fraude, confirme ou rejette
POST /scan/dolly	Contrôle rayon X	N'admet que les bagages déjà confirmés, renvoie la progression

POST /scan/soute	Compartiment avion	Marque le bagage en soute avant ou arrière
POST /scan/rush	Bagage restant	Marque pour réacheminement au prochain vol
POST /scan/load-all	Chargement groupé	Pousse en soute tous les bagages confirmés non rush
POST /scan/embarquement	Porte d'embarquement	Marque le passager embarqué, renvoie le reste à embarquer
GET /health	Supervision	Sonde de disponibilité, sans authentification

Authentification

L'API tourne avec la clé `service_role`, qui contourne la RLS. Elle authentifie donc elle-même chaque appel. Un hook Fastify valide le jeton Supabase de l'agent, vérifie son rôle, puis décore la requête avec l'identité vérifiée.

L'identité du scanneur est dérivée du jeton, jamais du corps de la requête. Un client ne peut donc pas se faire passer pour un autre agent dans la traçabilité.

```
Authorization: Bearer <access_token Supabase>
```

```
401 jeton absent, invalide ou expire
```

```
403 profil introuvable, ou role non autorise
```

a

4. Règles anti-fraude

Cinq règles de rejet, appliquées au scan d'une étiquette au tapis. Elles sont implémentées dans une fonction pure sans accès base, ce qui les rend testables indépendamment. La route assemble le contexte depuis la base, puis délègue la décision.

Règle	Condition	Conséquence
1	Aucun bagage pré-enregistré pour ce numéro de série sur ce vol	Rejet et alerte fraude
2	Le passager a déclaré 0 bagage	Rejet et alerte fraude
3	Quota atteint : confirmés supérieurs ou égaux aux déclarés	Rejet et alerte fraude
4	Étiquette déjà confirmée sur ce vol	Rejet simple, sans alerte
5	Le bagage appartient à un autre vol	Rejet simple, sans alerte

Les règles 4 et 5 ne déclenchent pas d'alerte : un double scan ou une erreur de porte n'est pas une fraude. Seules les règles 1 à 3 créent une entrée dans `fraud_alerts`, avec un anti-doublon par étiquette et par vol.

Responsabilité

L'agent bagage n'est jamais en faute : il scanne ce qui arrive sur le tapis. La fraude vient du comptoir d'enregistrement, où une étiquette a été imprimée sur un dossier sans bagage déclaré. Le système intercepte le colis avant la soute et alerte le superviseur pour intervention physique.

a

5. Cycle de vie d'un bagage

Chaque étape correspond à un poste de scan distinct et à un champ en base.

1. Check-in	boarding pass scanne	-> bagages pre-enregistres (is_confirmed = false)
2. Tapis bagages	etiquette scannee	-> is_confirmed = true, ou rejet anti-fraude
3. Dolly (rayon X)	etiquette scannee	-> on_dolly = true, si et seulement si confirme
4. Soute	etiquette scannee	-> soute = avant arriere
5. Chargement	action groupée	-> in_hold = true pour les non-rush
Rush	etiquette scannee	-> rush = true, reachemine au vol suivant

Le poste Dolly

Le dolly est le chariot qui amène les bagages contrôlés jusqu'à l'appareil. À la sortie du rayon X, l'agent scanne chaque bagage. Seuls les bagages déjà confirmés au tapis sont admis : tout bagage inconnu ou non enregistré est refusé. L'écran affiche la progression sous forme de compteur, et signale le dolly complet quand le nombre de bagages embarqués atteint le nombre de bagages enregistrés du vol.

a

6. Sécurité

Rôles

Rôle	Plateforme	Permissions
agent	Mobile uniquement	Scanner boarding pass et étiquettes
supervisor	Web	Dashboard, bagages, rapports, litiges
admin	Web	Tout, y compris créer et supprimer des comptes

Row Level Security

La RLS est active sur les huit tables. Les politiques s'appuient sur une fonction `auth_role()` en SECURITY DEFINER, qui lit le rôle sans déclencher la récursion des politiques de profils. Cette fonction doit rester exécutable par le rôle authenticated, sans quoi toutes les politiques cessent de fonctionner.

Création de comptes

L'inscription publique est désactivée. Les comptes sont créés uniquement par un administrateur, via une route serveur qui vérifie son rôle en base avant d'utiliser la clé d'administration. Un trigger alimente profils à partir des métadonnées, en restreignant le rôle aux trois valeurs valides.

Points de vigilance

- La clé `service_role` ne doit jamais être exposée côté client, uniquement dans l'API et les routes serveur.
- Les endpoints publics de suivi et de réclamation sont limités en débit par adresse IP.
- Les portails publics ne renvoient aucune donnée passager sans PNR ou numéro d'étiquette valide.

a

7. Temps réel et charge

Les tables passagers, bagage, fraud_alerts et flights sont publiées en realtime, avec REPLICA IDENTITY FULL pour que les événements de mise à jour portent aussi l'ancienne valeur des colonnes.

Comptage incrémental côté mobile

L'application mobile ne recharge pas les compteurs à chaque événement. Elle applique un delta local à partir de l'ancienne et de la nouvelle valeur reçues. Un scan effectué par un agent ne déclenche donc aucune requête chez les autres agents connectés.

Au démarrage, les statistiques de tous les vols du jour sont obtenues par une seule fonction SQL agrégée, au lieu d'une série de requêtes de comptage par vol.

```
select * from public.flight_stats_for_date(current_date);

flight_id | pax | bag_total | bag_ok | boarded
```

Cache du tableau des vols

Le portail public des vols sert la même liste à tous les visiteurs. Un cache mémoire de 20 secondes, avec déduplication des requêtes concurrentes, ramène la charge à une requête par date et par intervalle, quel que soit le nombre de visiteurs. Le filtrage par aéroport se fait côté client, ce qui préserve ce cache partagé.

a

8. Déploiement

Le monorepo est la source de vérité. Les cibles de déploiement sont des dépôts autonomes distincts, alimentés par recopie.

Dépôt	Contenu	Déploiement
Police	Monorepo complet	Source de vérité, ne déploie rien
API-POLICE	API autonome, bundle server.js	Hostinger, auto-deploy au push
police-web	Dashboard superviseur	Hébergement web
police-vols	Tableau des vols	Hébergement web
police-litige	Gestion des litiges	Hébergement web
police-tracking	Suivi bagage	Hébergement web

Modifier le monorepo ne propage rien automatiquement. Il faut recopier les sources dans le dépôt autonome concerné, reconstruire si nécessaire, puis pousser.

Application mobile

L'application mobile n'a pas de mise à jour à distance. Toute modification exige une reconstruction complète puis une réinstallation sur chaque PDA.

```
cd apps/mobile
eas build -p android --profile production
```

Conséquence à connaître : déployer une API qui exige quelque chose de nouveau du client casse les scans des PDA non encore mis à jour. Mettre à jour le mobile d'abord, l'API ensuite.

a

9. Commandes utiles

<code>npm install</code>	installer les dependances du monorepo
<code>npm run dev -w apps/web</code>	lancer un portail en developpement
<code>npm run typecheck -w @police/api</code>	verifier les types d'un paquet
<code>npm test -w @police/api</code>	executer les tests
<code>npx tsc --noEmit</code>	verifier un depot autonome
<code>node docs/capture.mjs --login</code>	captures d'ecran pour les manuels
<code>python docs/manuels_contenu.py</code>	generer les manuels d'utilisation
<code>python docs/technique_fr.py</code>	generer ce manuel technique